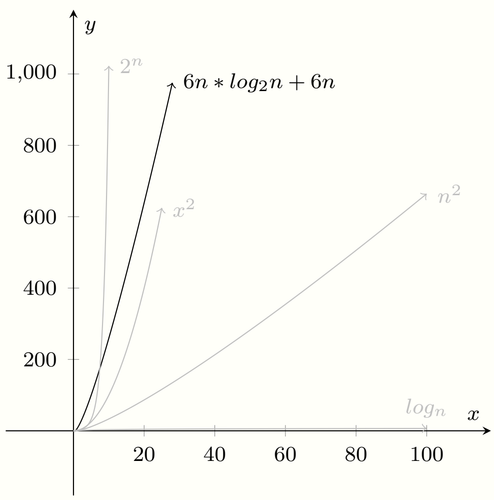
Merge Sort

1 Description

Recursive algorithm based on the divide and conquer strategy. It is based on two components: the function that split the given list recursively until it receives a single item list, and a merge function that is used on the recursive algorithm to merge two already sorted lists into a combined sorted one.

2 Time Complexity



3 Pseudocode

Input: $A = [A_1, A_2, ..., A_n]$
Output: $B = [B_1, B_2, ..., B_n]$

---

function MergeSort(A) ▷ Algorithm itself
    $n \leftarrow length(A)$;
    if $n \leq 1$ **then**:
        return A; ▷ Length 1, already sorted, return
    else
        $l \leftarrow [A_1, ..., A_{n/2}]$; ▷ Split left subarray
        $r \leftarrow [A_{(n/2)+1}, ..., A_n]$; ▷ Split right subarray
        $L \leftarrow$ MergeSort(l); ▷ Sort recursively
        $R \leftarrow$ MergeSort(r); ▷ Sort recursively
        $B \leftarrow$ Merge(L, R); ▷ Merge both subarrays
        return B;
    end if

end function

function MERGE(L, R)
    $n \leftarrow length(L) + length(R)$; ▷ Full length of resulting array
    $B \leftarrow [\ ]$; ▷ Initialize empty array
    $i \leftarrow 0$;
    $j \leftarrow 0$;

    for $k \leftarrow 0$ to n **do** ▷ Iterate full length of both subarrays
        if $i \leq length(L)$ **and** $j \leq length(R)$ **then** ▷ Both within range
            if $L[i] \leq R[j]$ **then**: ▷ Left lower, set it
                $B[k] \leftarrow L[i]$;
                $i \leftarrow i + 1$;
            else if $L[i] > R[j]$ **then**: ▷ Right lower, set it
                $B[k] \leftarrow R[j]$;
                $j \leftarrow j + 1$;
            end if
        else if $i + 1 \leq length(L)$ **then**: ▷ Only left within range, set it
            $B[k] \leftarrow L[i]$;
            $i \leftarrow i + 1$;
        else if $j + 1 \leq length(R)$ **then**: ▷ Only right within range, set it
            $B[k] \leftarrow R[j]$;
            $j \leftarrow j + 1$;
        end if
    end for
    return B;
end function

Antonio Díaz Correa
1st September 2024